SWE40006

Software Deployment and Evolution

Deployment Portfolio Task 3

TASK 3 | AZURE APP SERVICE | HD ATTEMPT

Individual assignment report covering Task 3.1 (Pass), Task 3.2 (Credit) and Task 3.3 (High Distinction).

STUDENT / UNIT FIELD	VALUE
Full name	Nguyen Huyen Minh Nhat
Student ID	105550225 - SWH02356
Teaching period / semester	Fall 2026
Unit	SWE40006 - Software Deployment and Evolution
Declared level	Task 3.3 - High Distinction (including 3.1 and 3.2)
Due date	19 October 2026, 23:59
Submission date	18 October 2026

DEPLOYED APPLICATIONS

SUB-TASK	APPLICATION	RESULT
3.1 / Pass	Existing ASP.NET Core sample	.NET 10 on Azure / online
3.2 / Credit	DeployLens - C# edition	.NET 10 on Azure / stopped as required
3.3 / HD	DeployLens - PHP edition	PHP 8.4 on Azure / online

1. Purpose and scope

For this task, I deployed an existing ASP.NET Core sample to Azure, then built and deployed two versions of DeployLens, one in C# and one in PHP. I completed the Pass and Credit steps as part of my attempt at Task 3.3 (High Distinction).

1.1. Mapping to the assessment brief

TASK	REQUIRED WORK	EVIDENCE IN THIS REPORT
3.1 / Pass	Azure account; VS Code; existing app deployed to Azure	Azure resource, publish result and public Counter page
3.2 / Credit	Develop C# app; deploy; deactivate	DeployLens C# source overview, running Azure assessment, Stopped state and 403 response
3.3 / HD	Install PHP; develop web app; deploy to Azure	PHP version, DeployLens PHP app, Azure deployment logs and public health endpoint

1.2. Selected architecture

I kept the three Web Apps in the same Azure resource group, rg-swe40006-task3. The ASP.NET sample and DeployLens C# share a Windows Free (F1) App Service Plan. The PHP version runs on a separate Linux Free (F1) plan, as the Windows plan could not host the selected PHP runtime.

APPLICATION	RUNTIME / PLAN	INTENDED FINAL STATE
SWE40006-Task3-Sample	.NET 10 / Windows Free F1	Running
SWE40006-Task3-CS	.NET 10 / Windows Free F1	Stopped (deactivated)
SWE40006-Task3-PHP	PHP 8.4 / Linux Free F1	Running

1.3. Development and deployment workflow

I tested each application locally before creating its Azure Web App. I then deployed the required files with the Azure App Service extension in VS Code and tested the public site. Once I had verified the C# version, I stopped only that Web App to meet the deactivation requirement. I left the PHP version running for verification.

2. Task 3.1 - Existing ASP.NET Core application

2.1. App selection and local preparation

I chose Microsoft Azure Samples' existing dotnetcore-docs-hello-world application for Task 3.1. I kept it as an existing sample rather than claiming to have developed it. On my laptop, I used VS Code, Git and .NET SDK 10.0.401.

On my first local run, the page had no styling and the console reported missing static assets. I found that the app was running directly from source in Production mode. After restarting it in Development mode, the layout loaded properly. I then prepared the deployment files with dotnet publish -c Release -o .\publish.

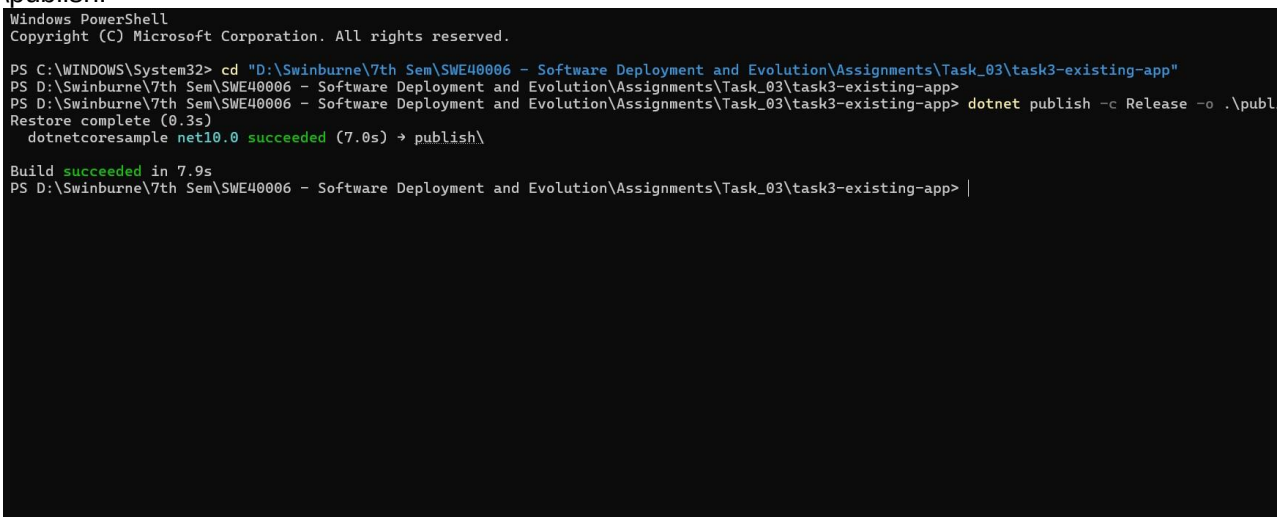


Fig. 1. Release publish output for the existing ASP.NET Core sample.

2.2. Azure deployment

I created a Windows Web App named SWE40006-Task3-Sample on the .NET 10 runtime, using the Free F1 tier. I deployed the published folder from VS Code and checked that the Azure deployment completed successfully.

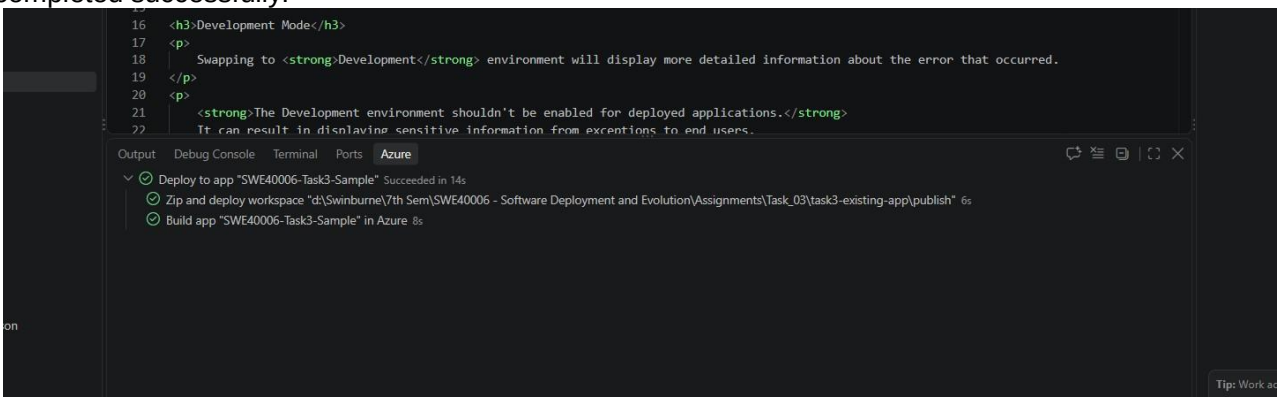


Fig. 2. VS Code Azure App Service deployment succeeded for SWE40006-Task3-Sample.

2.3 Public verification

The deployed sample loaded over HTTPS with the expected navigation and styling. I opened the Counter page and clicked the button until the counter reached 20. This confirmed that the live application was responding to user interaction.

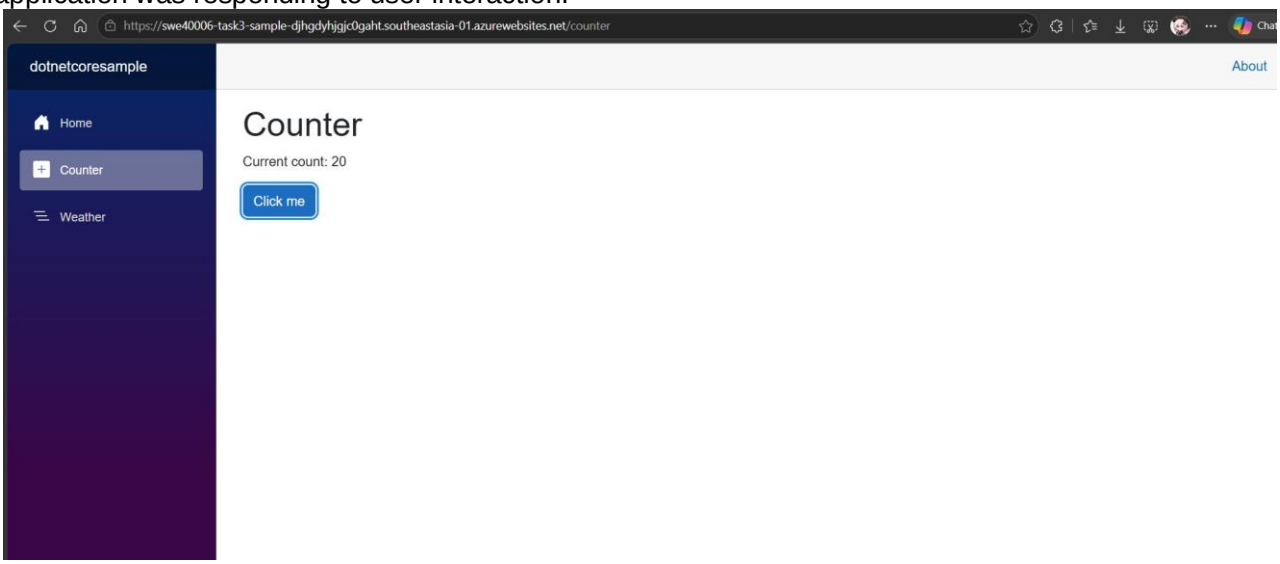


Fig. 3. Public Counter page running on Azure App Service.

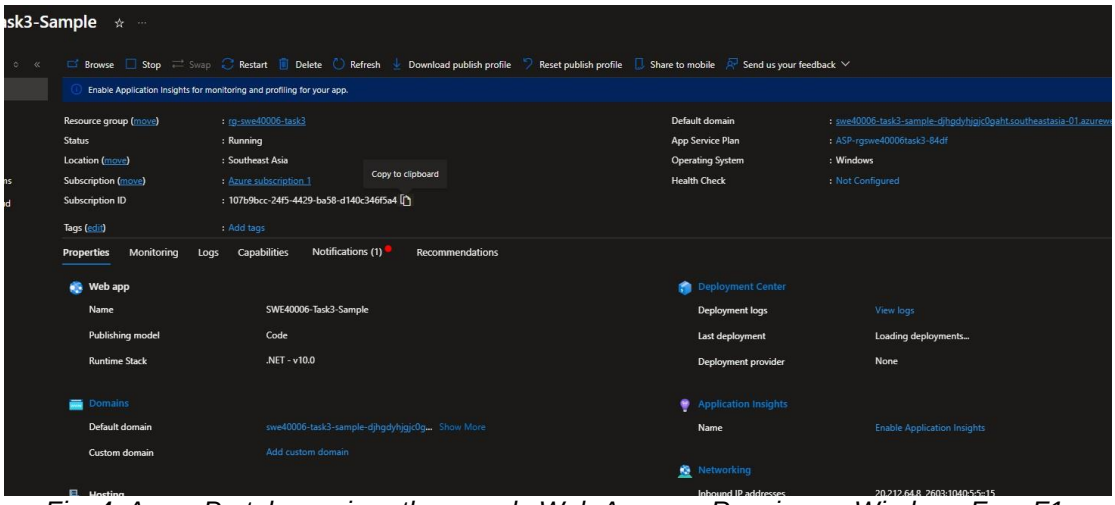


Fig. 4. Azure Portal overview: the sample Web App was Running on Windows Free F1.

Existing application source (Microsoft Azure Samples): https://github.com/Azure-Samples/dotnetcore-docs-hello-world
Public sample URL: https://swe40006-task3-sample-djhgdyhjgjc0gaht.southeastasia-01.azurewebsites.net

3. Task 3.2 - C# application: DeployLens

3.1. Development and server-side logic

For Task 3.2, I built DeployLens in C# using ASP.NET Core Minimal APIs on .NET 10. It has a release form, six checks to complete before deployment, result cards and a JSON export option. The backend includes GET /api/health and POST /api/analyze.

The C# endpoint checks the release name, version, environment, change type and checklist values. It calculates risk from the unchecked controls, with additional penalties for production releases and certain change types. The server returns the score, status and findings, rather than leaving the calculation entirely to JavaScript.

3.2. Build and deployment evidence

I built and published the C# edition in Release mode, then deployed it from VS Code to SWE40006-Task3-CS. The deployment finished without an error.

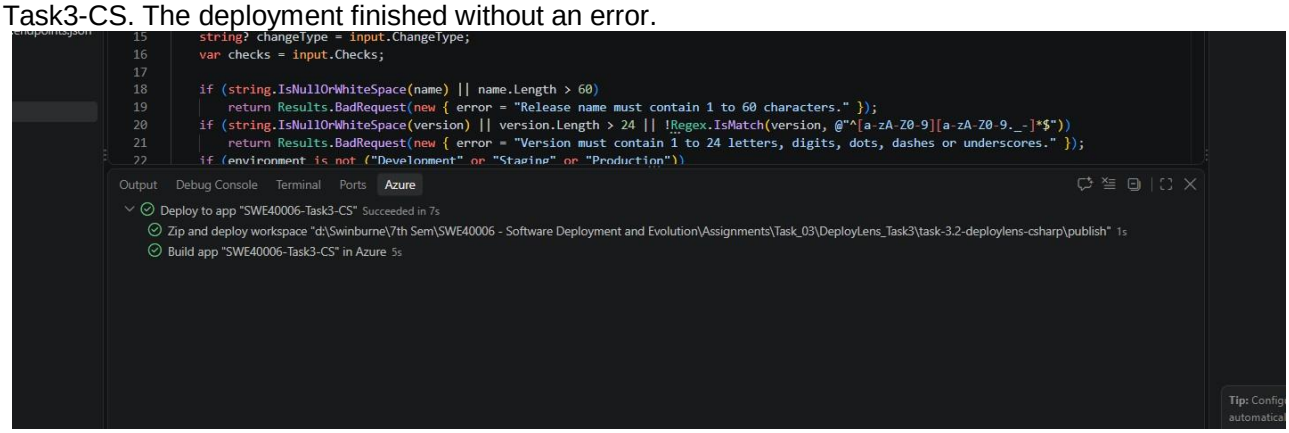


Fig. 5. Successful Azure push deployment of the DeployLens C# edition.

3.3. Functional verification

On the Azure website, the DeployLens interface loaded with its CSS and fonts. I loaded the example release and checked how the backend responded when staging verification and the rollback plan were left incomplete.

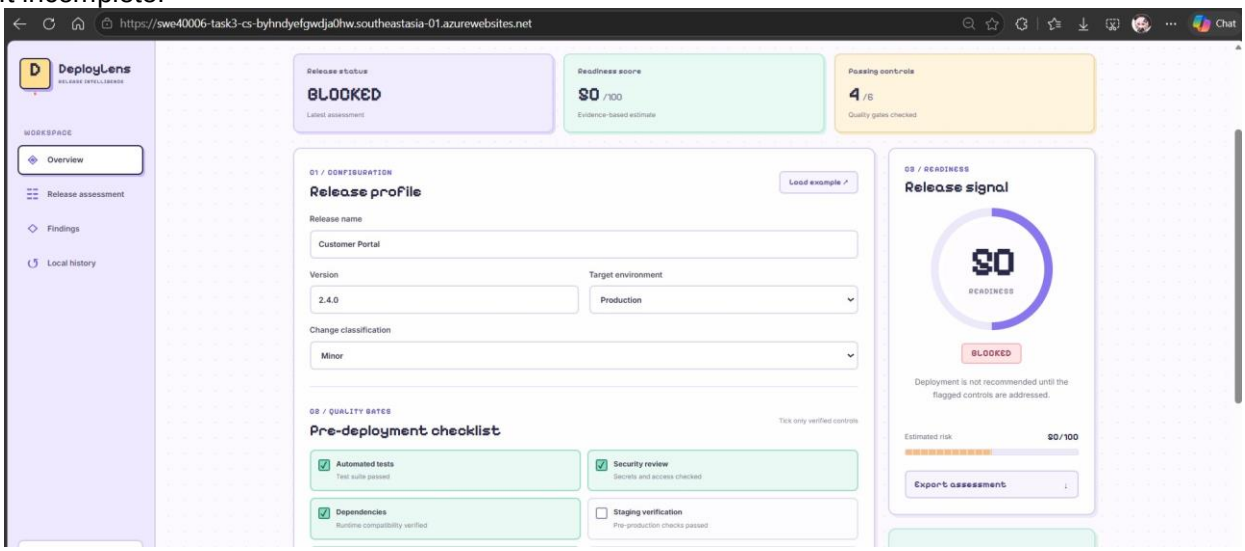


Fig. 6. DeployLens C# assessment: BLOCKED, 50/100 with 4/6 checks passed.

3.4. Ready result and export

After checking the two remaining controls, I ran the assessment again. The result changed to READY with a score of 86/100. I downloaded the JSON export and checked that it included the release details, score, risk, status and assessment time.

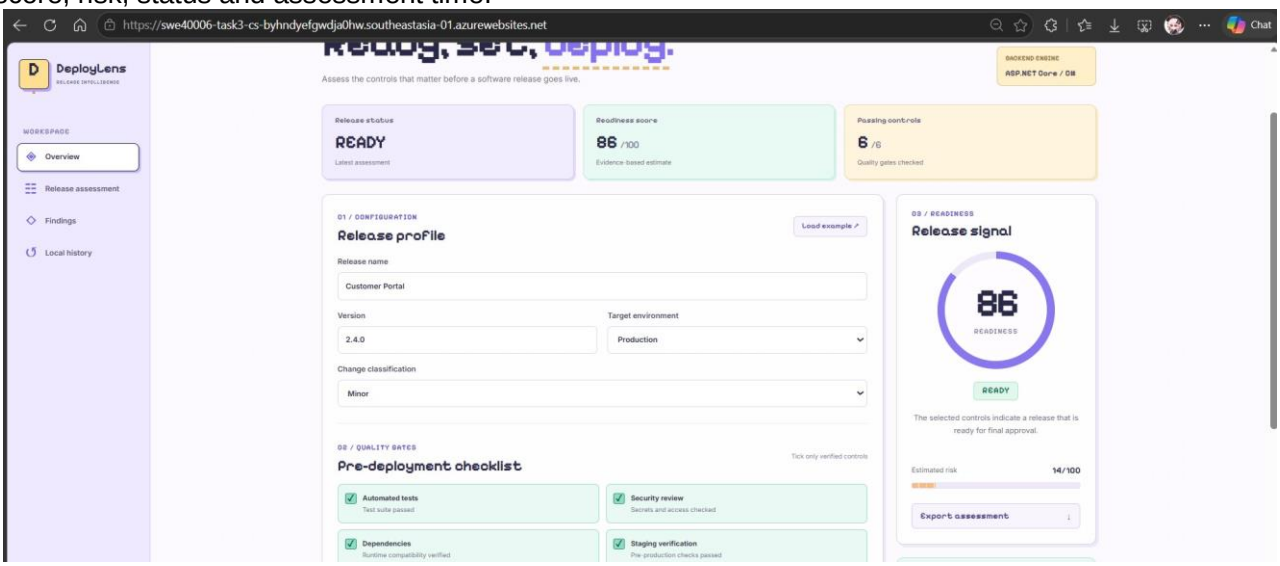


Fig. 7. DeployLens C#: READY result with 86/100 and 6/6 checks passed.

I checked /api/health separately and received status: ok with the service name DeployLens C#. I completed these checks before stopping the app.

3.5. Deactivation (required by Task 3.2)

I used the Stop action for SWE40006-Task3-CS in Azure Portal after the tests were finished. The status changed to Stopped, and opening the public address returned Azure's expected 403 message: "This web app is stopped." I kept screenshots of both results.

3.6 Deactivation evidence

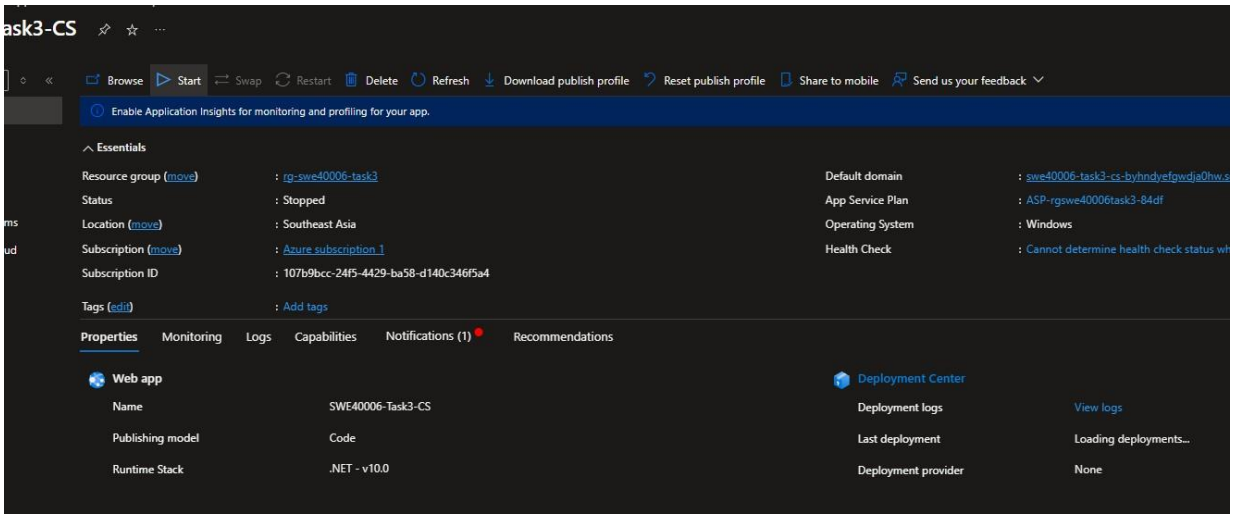


Fig. 8. Azure Portal showing the C# Web App in the Stopped state.

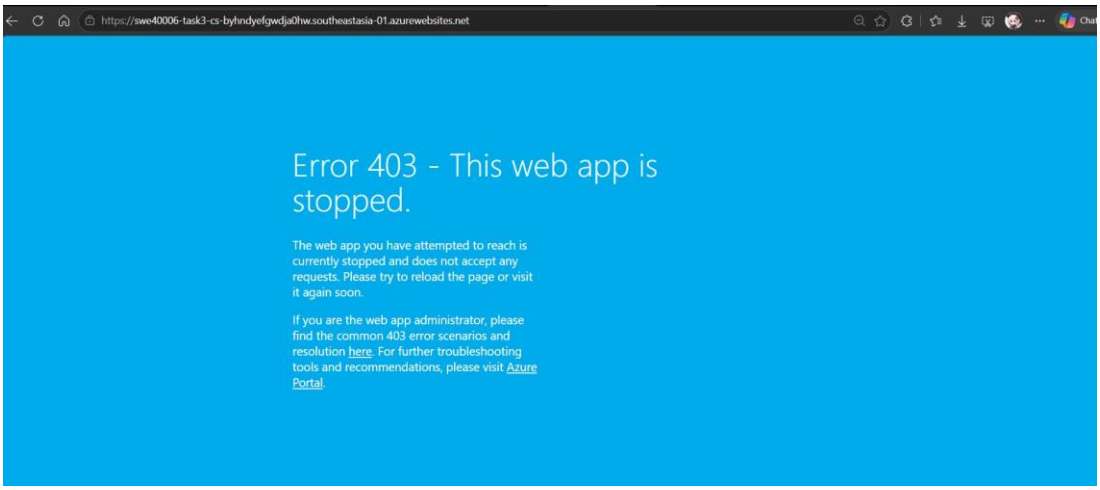


Fig. 9. Expected HTTP 403 response after C# Web App deactivation.

4. Task 3.3 - PHP application: DeployLens

4.1. PHP installation and implementation

I installed PHP on Windows and confirmed the installation using php -v. PowerShell reported PHP 8.4.26 (x64, Non Thread Safe). I then built a PHP version of DeployLens with its own server-side analysis logic, independent of the C# service.

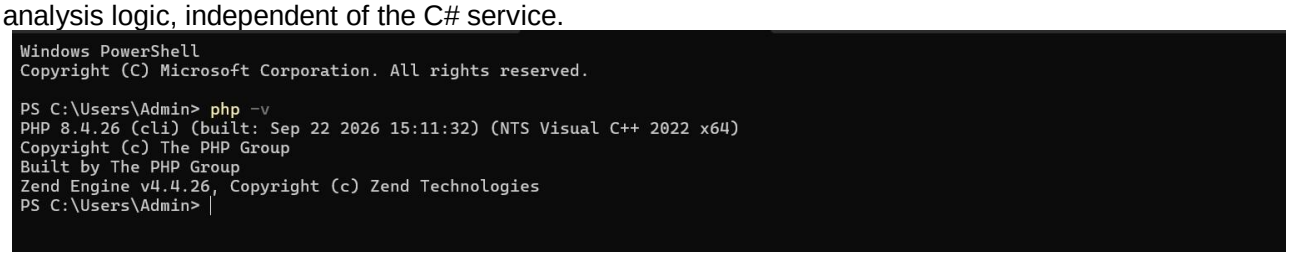


Fig. 10. PowerShell output verifying PHP 8.4.26 installation.

The PHP backend accepts health and analysis requests, checks the submitted JSON, evaluates the six controls and returns a result. I kept the assessment rules and interface consistent with the C# version. The PHP page can also export the assessment as JSON.

4.2. Azure Linux App Service

I created SWE40006-Task3-PHP in the existing resource group and selected PHP 8.4 on Linux with a Free F1 plan. Unlike the C# version, I deployed the PHP source folder directly from VS Code without running dotnet publish.

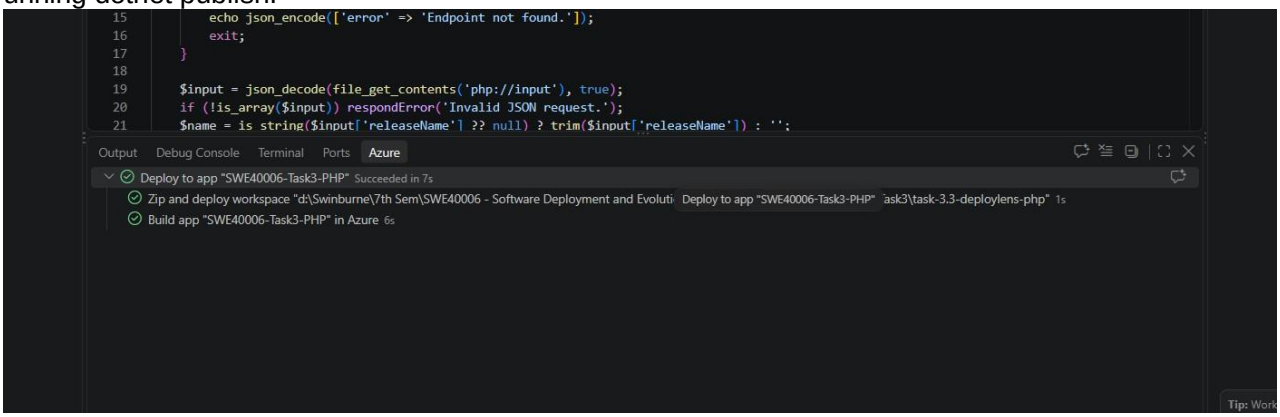


Fig. 11. VS Code confirmation of successful PHP source deployment to Azure.

4.3. Public-site functionality

The deployed PHP website loaded with its fonts, CSS and navigation. I tested the example release with incomplete staging and rollback controls; the app returned BLOCKED and a score of 50/100. After selecting the missing controls, the result changed to READY and 86/100.

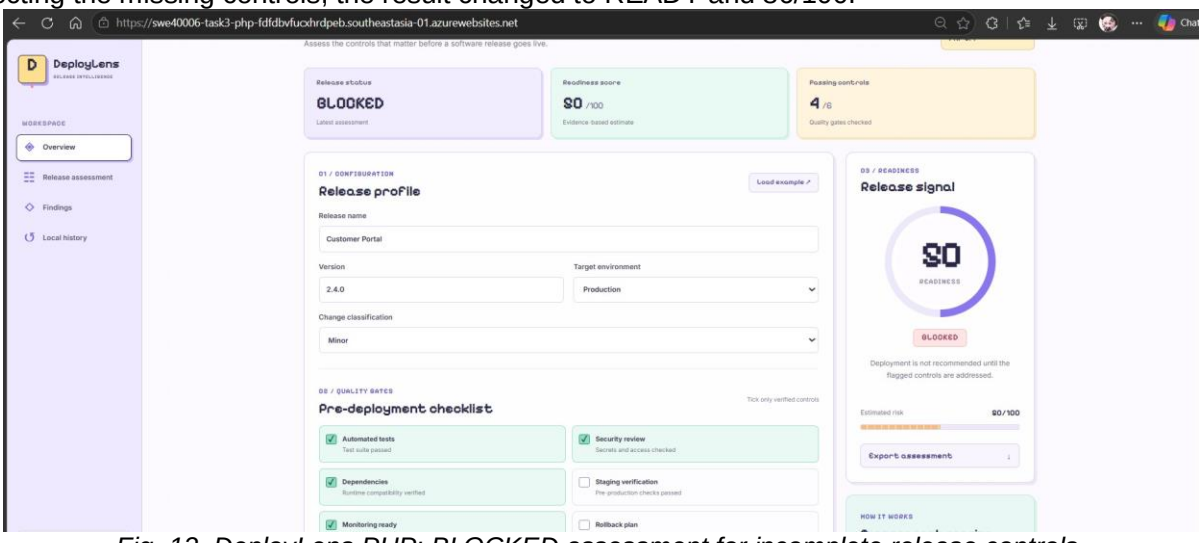


Fig. 12. DeployLens PHP: BLOCKED assessment for incomplete release controls.

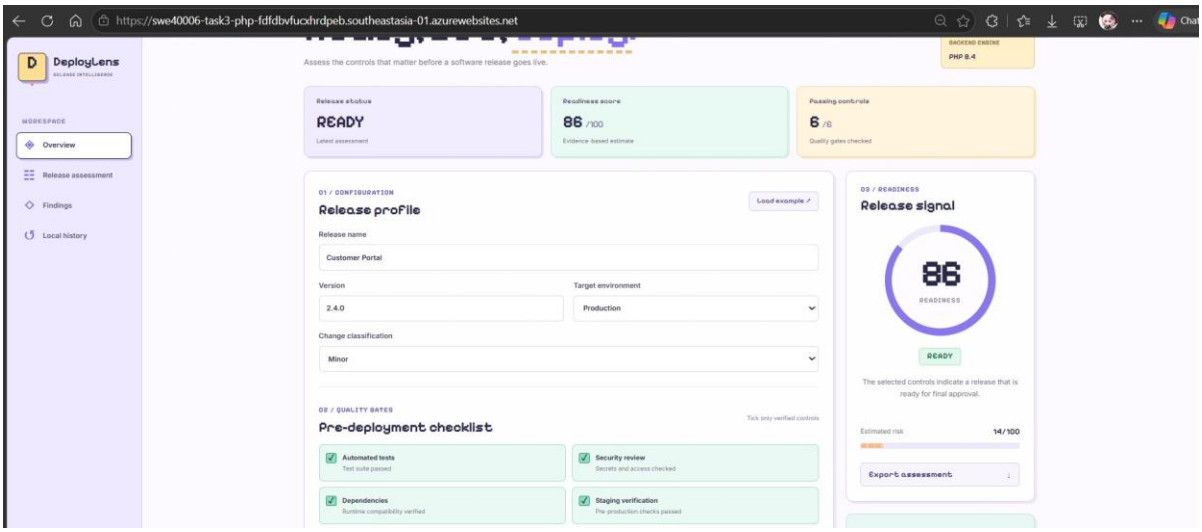


Fig. 13. DeployLens PHP: READY assessment, 86/100 with all six controls selected.

The PHP JSON export recorded READY, a score of 86, and a risk value of 14. This matched the C# export for the same example release.

4.4. Azure runtime and deployment verification

The live PHP health endpoint returned status: ok and service: DeployLens PHP. In Azure Portal, Deployment Center > Logs showed a successful active deployment created through the VS Code Azure Tools extension (push deployment). CI/CD was not configured because deployment was performed manually.



Fig. 14. Live PHP health endpoint responding through the Azure public domain.

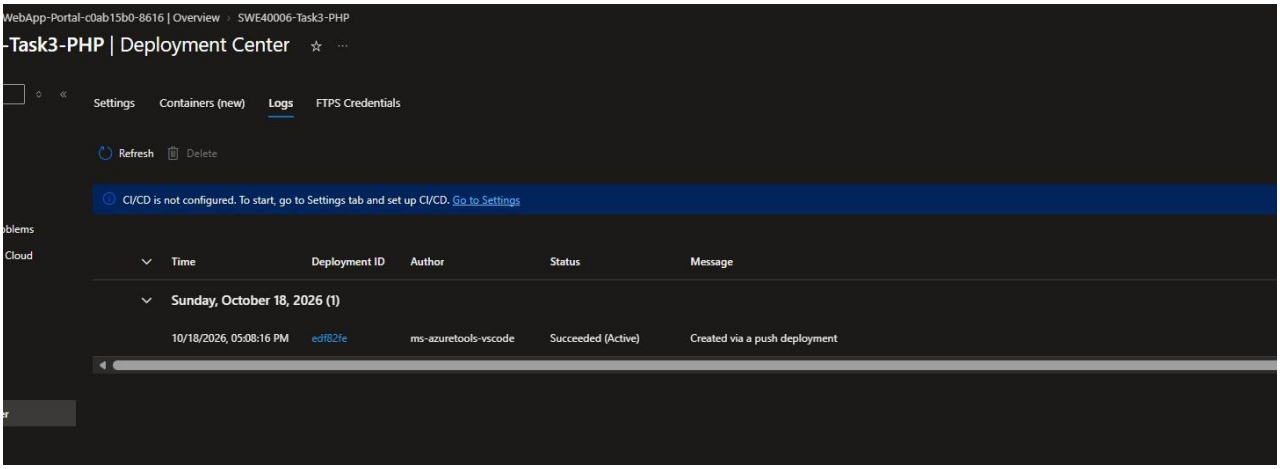


Fig. 15. Azure Deployment Center Logs: successful and active PHP push deployment.

Public PHP URL: https://swe40006-task3-php-fdfdbvfucxhrdpeb.southeastasia-01.azurewebsites.net
Health check: https://swe40006-task3-php-fdfdbvfucxhrdpeb.southeastasia-01.azurewebsites.net/?api=health

5. Error investigation and troubleshooting

5.1. Missing static assets during ASP.NET local testing

The first launch of the existing ASP.NET Core sample displayed unstyled HTML and the Blazor error banner. PowerShell logs showed the app running in Production mode from its source directory and warnings that static web assets were not enabled. FileNotFoundException entries referred to generated or framework assets such as dotnetcoresample.styles.css and blazor.web.js.

I first checked the hosting-environment output instead of creating placeholder files for assets that should be generated or served by the ASP.NET runtime. I stopped the local process and started it again with ASPNETCORE_ENVIRONMENT and DOTNET_ENVIRONMENT set to Development. The UI then loaded with the correct styling and the application built successfully. The production deployment was subsequently made from the published Release output.

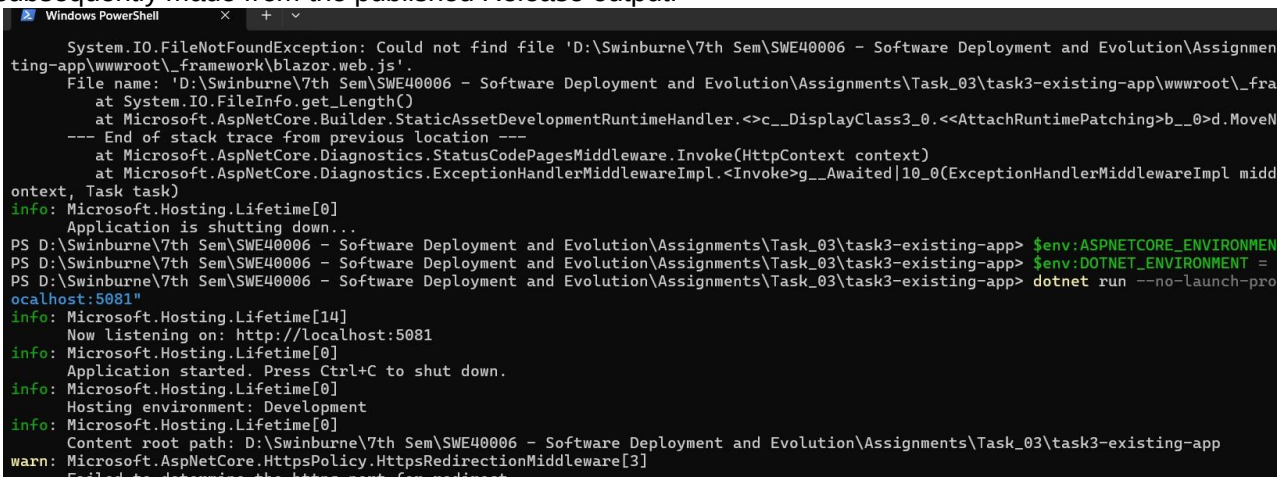


Fig. 16. PowerShell log covering the initial asset failure, environment correction and successful local build.

5.2. Azure diagnostics and messages

OBSERVATION	INTERPRETATION / STATUS / NOTE	OUTCOME
Basic authentication disabled notice	Informational notice on resource setup. I retained the default setting and used the VS Code deployment extension; I did not treat it as a deployment failure.	VS Code push deployments succeeded.
Deployment Center: CI/CD not configured	Expected because I used manual push deployment, not GitHub continuous deployment.	PHP deployment log showed Succeeded (Active).
403 after stopping C#	Expected result of deliberate deactivation, not an application bug to fix.	Azure Portal showed Stopped and the public URL returned the stopped-app page.
PHP runtime health check	Checked independently of deployment status to confirm the application answered web requests.	Public API returned status: ok.

6. Verification results and public links

6.1. Evidence summary

REQUIREMENT	RESULT / VERIFICATION
Task 3.1: existing web app	Successfully published and deployed; interactive Counter page verified on Azure.
Task 3.2: develop C#	DeployLens C# implementation and JSON API validated using public Azure deployment.
Task 3.2: deactivate app	Azure Stopped state and expected 403 browser response captured.
Task 3.3: install PHP	PHP 8.4.26 confirmed from php -v.
Task 3.3: develop and deploy PHP	DeployLens PHP deployed successfully; web UI, scoring and health endpoint verified.
Supporting assessment data	C# and PHP JSON exports both record READY, score 86/100, risk 14/100.

6.2. URLs for lecturer verification

SERVICE	PUBLIC ADDRESS / STATUS
Task 3.1 - existing ASP.NET sample	https://swe40006-task3-sample-djhgdyhjgjc0gaht.southeastasia-01.azurewebsites.net [Running when captured]
Task 3.2 - DeployLens C#	https://swe40006-task3-cs-byhndyefgwdja0hw.southeastasia-01.azurewebsites.net [Stopped deliberately]
Task 3.3 - DeployLens PHP	https://swe40006-task3-php-fdfdbvfucxhrdpeb.southeastasia-01.azurewebsites.net [Running when captured]

6.3. Source availability

Existing application source (upstream): https://github.com/Azure-Samples/dotnetcore-docs-hello-world
Coursework repository: https://github.com/NhatNguyen2407/Swinburne/tree/main/SWE40006_Software_Deployment_and_Evolution/Assignments/Task%203 (SWE40006 / Assignments / Task 3)
The repository is the source location for this coursework. I kept the Microsoft sample separate from the C# and PHP DeployLens projects so each sub-task can be reviewed on its own.

6.4. Included supporting files

The project folder also contains cs-deploylens-2.4.0-assessment.json and php-deploylens-2.4.0-assessment.json. These JSON exports support verification of the sample READY assessment and are not substitutes for the required deployment report.

7. Reflection and conclusion

Completing Task 3 showed me the difference between running a web application locally and making it available on a managed cloud platform. Local execution alone did not satisfy the task; I had to configure Azure resources, deploy the correct output, check public URLs and confirm the application still behaved as expected.

The existing ASP.NET sample was useful for learning the initial workflow. I then used that process for a C# application with my own backend logic, before stopping the Azure service as required. Finally, I deployed an independent PHP version of the same DeployLens product on Linux. This also highlighted why the Windows and Linux hosting plans had to be handled separately.

The main technical issue I encountered was loading ASP.NET static assets while running the sample from source in Production mode. Reviewing the terminal log and adjusting the local environment resolved the issue. On Azure, the recorded deployments completed successfully; the 403 shown for the C# URL was the expected effect of my deliberate deactivation.

The results shown in this report cover the requirements of Task 3.1, 3.2 and 3.3. The PHP app remains available for testing, while the C# app was stopped after its deployment and verification had been recorded.

References / source materials

1. SWE40006, “Deployment Portfolio Task 3 (10 Points)” - assignment brief and rubric supplied via Canvas.
2. Microsoft Azure Samples, dotnetcore-docs-hello-world - source repository for the existing Task 3.1 application.
3. Local Task 3 source folder - project files, screenshots and C#/PHP assessment exports used for the tests in this report.